JOMHACK
EMPOWERING INNOVATION

```
> bootcamp.learn()
```

JOMHACK
EMPOWERING INNOVATION

JomCode Coding Bootcamp

CHAPTER 10

Files, JSON, virtual environments and configuration

`learn / type / run / explain`

Plain meaning and technical explanation

PLAIN MEANING

Files persist data. An isolated environment keeps a project's packages together.

TECHNICAL EXPLANATION

with closes resources. JSON serializes common data types. venv isolates dependencies. Environment variables supply settings without hardcoding secrets.

REAL-WORLD USE

Prepare the configuration and dependency workflow used by the backend.

QUICK TIP

Keep .env, .venv and node_modules out of Git. Commit .env.example with placeholders.

INSTALL / RUN

Windows: py -m venv .venv; .venv\Scripts\python.exe workbook.py. macOS: python3 -m venv .venv; .venv/bin/python workbook.py. No packages.

Worked example 1: JSON round trip

workbook.py

```
import json
post = {"title": "Hello", "image": None}
encoded = json.dumps(post)
decoded = json.loads(encoded)
print(encoded)
print(decoded == post)
```

PRO TIP UUID and datetime need conversion before JSON serialization.

Run and explain example 1

EXPECTED RESULT

{"title": "Hello", "image": null}
True

WHY IT BEHAVES THIS WAY

JSON null maps to Python None. JSON booleans use lowercase true and false.

Worked example 2: A temporary text file

workbook.py

```
from pathlib import Path
from tempfile import TemporaryDirectory
with TemporaryDirectory() as folder:
    path = Path(folder) / "draft.txt"
    with path.open("w", encoding="utf-8") as file:
        file.write("Hello, JomCode")
    print(path.read_text(encoding="utf-8"))
```

PRO TIP Use UTF-8 explicitly for portable text files.

JOMHACK
EMPOWERING INNOVATION

Run and explain example 2

EXPECTED RESULT

Hello, JomCode

WHY IT BEHAVES THIS WAY

The file closes before reading. The temporary directory is removed when the block ends.

Worked example 3: Configuration defaults

workbook.py

```
01    import os
02    origin = os.getenv("FRONTEND_ORIGIN", "http://localhost:5173")
03    print(origin)
04    print(bool(os.getenv("SUPABASE_DB_URL")))
```

PRO TIP Use pydantic-settings in the application to load .env.

Run and explain example 3

EXPECTED RESULT

Without environment overrides:
http://localhost:5173
False

WHY IT BEHAVES THIS WAY

getenv() reads the process environment. It does not automatically load a .env file.

Worked example 4: Check the active interpreter

workbook.py

```
import sys
from importlib.metadata import PackageNotFoundError, version
print(sys.executable)
try:
    print(version("fastapi"))
except PackageNotFoundError:
    print("FastAPI is not installed in this interpreter")
```

PRO TIP Windows can use .venv\Scripts\python.exe directly without activating PowerShell scripts.

Run and explain example 4

EXPECTED RESULT

Interpreter path, followed by the installed FastAPI version or a clear missing-package message.

WHY IT BEHAVES THIS WAY

python -m pip installs into the interpreter you invoke. A different terminal can select a different environment.

Exercise 1: Store a draft

YOUR TASK

Write a post dictionary to JSON in a temporary directory, read it and verify the title.

CHECK THESE CASES

Unicode title survives. Missing image remains None.

HINT AFTER A TRY

Use json.dumps(..., ensure_ascii=False) and UTF-8.

Exercise 2: Required setting

YOUR TASK

Write a function that reports a missing setting without printing its value.

CHECK THESE CASES

Missing or empty value raises ValueError. A present value returns successfully.

HINT AFTER A TRY

Read with os.getenv and test truthiness.

Checkpoint before moving on

01 Run and explain all four worked examples. Match their actual results to the expected behavior.

02 Attempt both exercises before reading the solutions. Verify every acceptance case.

03 Explain how this chapter supports Postlight: Prepare the configuration and dependency workflow used by the backend.

Official reference